COURSE NAME -

TypeScript

Introduction to TypeScript -

1. TypeScript is a programming language, which was developed by Microsoft in the year 2012.
2. TypeScript is a superset of JavaScript which adds more features like static typing, error detection, and code readability.
3. JavaScript is flexible but sometimes it creates bugs because it does not check the type of the data.

   // for example -

   JavaScript:

```
let age = "20";

console.log(age) // No error
```

   TypeScript:

```
let age:Number;

age = "20"

console.log(age) // TypeError
```

Why TypeScript ? -

1. We use typescript because it detects errors at compile time.
2. TypeScript improves code readability and maintainability.
3. It provides strong typing and smart features like auto completion, auto suggestion, error checking.
4. TypeScript makes large projects easy to manage.
5. TypeScript helps testers to write reliable automation scripts.

Why not only JavaScript ? -

1. In JavaScript, most errors are found only when the program runs.
2. Using JavaScript, it is hard to handle big projects.
3. JavaScript does not have type safety (it doesn't check data types).
4. It becomes difficult to maintain large and complex automation frameworks in JavaScript.

// To solve these problems, we use TypeScript.

// TypeScript gives compile-time checking and type safety, so we can find errors early and manage projects easily.

History of TypeScript -

1. TypeScript was introduced in 2012, created by Microsoft and designed by Anders Hejlsberg .
2. The reason for the creation of TypeScript was that JavaScript is used for building large web applications but it had problems with

that, so to overcome those problems TypeScript was introduced which was more powerful, safer and maintainable as compared to JavaScript.

TypeScript Growth -

TypeScript become more popular because :

1. It supports the latest JavaScript features.
2. It makes code easy to read and understand.
3. It helps to write better quality code.
4. It helps testers write clean and readable automation scripts.
5. TypeScript works with frameworks like Angular, React, and Node.js. // When Angular adopted TypeScript, it became more popular.

Version of TypeScript -

// TypeScript updates its version regularly.

1. **version 1:** The first version of TypeScript was released in the year 2012, which was the basic static typing.
2. **version 2:** In this version , there was better type checking and optional parameters.
3. **version 3:** In this version, they improved performance and introduced advanced types.

4. **version 4:** in this version, they introduced faster compilation and powerful type features.
5. **version 5:** In this version, they enhanced typescript performance and also supported the latest features of JavaScript.
   // Currently we are using version 5.9 series.

What is a Transpiler ? -

A transpiler is a tool that converts code written in one programming language to another programming language.

Why do we need a Transpiler ? -

1. As browsers don't support the new JavaScript feature so for that we need a transpiler.
2. TypeScript cannot run on browsers directly.
3. We need a transpiler to convert the code that can work everywhere.

The Flow of Transpiler -



What is TSC ? -

1. TSC stands for TypeScript Compiler.
2. It is a command - line tool provided by TypeScript, to convert TypeScript code into JavaScript code.
3. It checks whether there is any error during compilation.
4. As browsers and Node.js cannot execute TypeScript code directly, TSC ensures that TypeScript code is validated, Type - checked and converted into a standard Javascript code that can run in every environment.

Installation process of TypeScript -

1. First create a new folder (Titled as TypseScript).
2. Open that folder and open cmd.
3. After opening cmd, check if your desktop / laptop is having node version or not (to check node version simply write node --version).
4. If your desktop / laptop is not having node then install node.js by downloading it through their official website https://nodejs.org/en/download.
5. After downloading proceed with the further steps.

6. If node is present in your desktop / laptop check npm (node package manager) (to check npm simply type npm -v command in the command prompt).
7. Install typescript package in your folder to install typescript package type (npm install -g typescript)
8. To check whether the compiler is properly installed in your desktop / laptop just type (tsc -v).
9. After that, open VS Code on your desktop / laptop.
10. After that, create two files, one app.ts and another file as index.html.

// To Compile a TSC file, just write in the terminal / cmd (tsc file_name.ts).

// To run the code in the terminal, we use (node file_name.ts).

Type Annotation -

1. A TypeAnnotation is a process in which we explicitly define the datatype of a variable, function, parameter return or an object in TypeScript.
2. In Type Annotation, it tells the compiler what kind / type of data will be present and helps to catch the error at the compile time

.

// Syntax for Type Annotation -

Variable_name : datatype = value; { The basic formula for type annotation is to add a colon(:) followed by datatype after that value.}

// Example - let age : number = 20;

```
let a : number;
a = 500;
console.log(a);
```

Why Type Annotation is needed ? -

1. Type Annotation prevents run time error.
2. It improves code readability.
3. Type Annotation provides compile - time safety.
4. It makes the code self - documented.

Common Type of Annotation -

1. **Primitive Types -** It is used for basic data like string, number, boolean, null and undefined.
2. **Array -** It specify that an array should only contain a specific type. (for example - let score:number[]=[10,20])
3. **Object -** It explicitly defines the shape of an object. (for example - let user:{name:string,age:number}={name = “Ankita”,age = 25})

When to use Type Annotation ? -

1. When a variable is declared without an initial value.
2. Functions have complex return logic.

Type Inference -

Type inference in TypeScript means the compiler automatically understands the data type based on the value you give, so you don't need to write the type manually, and your code stays safe.

Why is Type Inference useful ? -

1. Type Inference reduces the need for explicit type annotation.
2. It keeps the code clean and readable.
3. It maintains strong type safety.
4. It helps to catch an error at compile time.
5. It improves developer productivity.

   //Syntax -

   Let variable_name = value; { here, let is variable declaration type, variable_name is the name given to the variable and value is the data assigned to that variable.}

```
let data = 600
compiler version :- let data : number =
"TypeScript"
data = 100
console.log(data);

let a;      //! with respect to type inference
a = 10;
a = "TypeScript"
```

```
console.log(a);

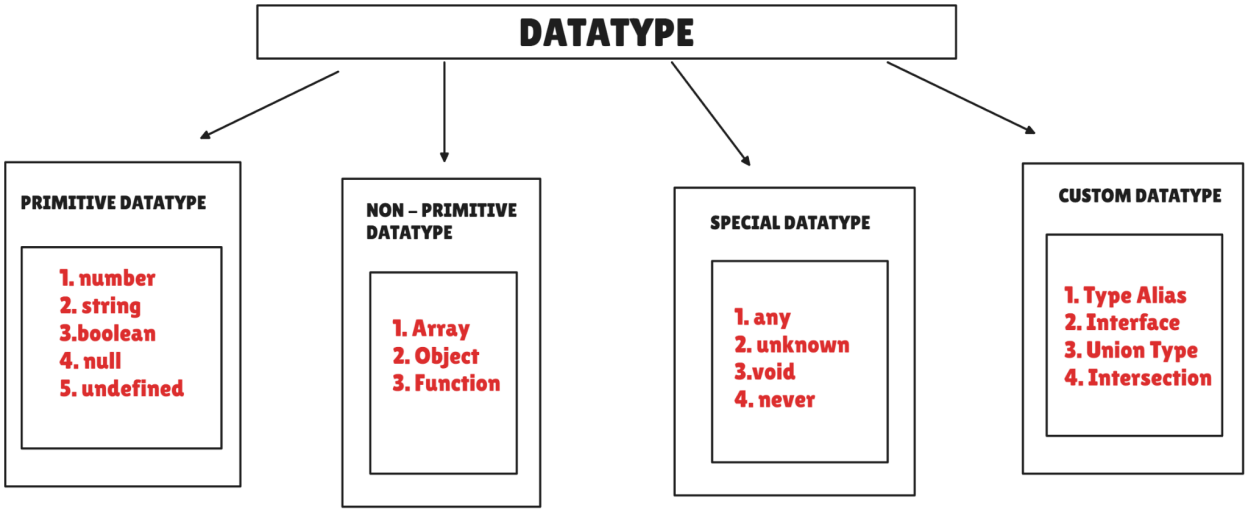
let a : number;    //! with respect to type
Annotation
a = 500;
console.log(a);
```

Difference between Type Annotation and Type Inference

type annotation	type inference
1. type annotation explicit provides the datatype to a particular variable. 2. the datatype is provided by the developer. 3. it is more controlled over declared type.	1. in type inference we are not providing explicitly the datatype to the variable. 2. the datatype is provided by the compiler. 3. it is less controlled over the declared type.

DataType in TypeScript -

Typescript datatype types us to define what kind of data is being stored in a variable.



In TypeScript, there are 4 types datatype in typescript : -

1. **Primitive datatype -** Primitive datatype in typescript stores single values that are immutable in nature that means we cannot change the type of the value.

   There are different types in primitive datatype -

   a. **Number:** number datatype is used to store numeric values (integer value and decimal value).

   For example -

```
let num : number = 10; //! --> integer value
console.log(num);

let num : number = 15.60;   //! --> decimal value
console.log(num);
```

```
let num : number = -250;    //! --> negative value
console.log(num);
```

b. **String:** String datatype is used to store textual data (sentence, character, word). For representing string there are three ways :-

- Single quotes -

  For example -

  ```
  let a : string = 'hi everyone, a very good morning after the session i will be sending the notes.'
  console.log(a);
  ```

- Double quotes -

  For example -

  ```
  let b :string = " let's go to a party."
  console.log(b);
  ```

- Backticks (temporal literal) - A string interpolation is a concept in which we are accessing a value which is

assigned to a variable into a sentence. where we are using `` and ${}.

For example -

```
let age : number = 30
console.log(`my age is ${age}`);
```

c. **Bigint :** A bigint is a primitive datatype which is used to store a large number exceeding the limit of a normal number value and it is represented with the help of a letter at the n.

For example -

```
let num : bigint =
123456789000523698714581215221n
console.log(num);
```

d. **Boolean:** Boolean datatype is used to store true or false values. There are only two possible values either true or false.

For example -

```
let isPresent : boolean = true;
console.log(isPresent);
```

e. **Null:** Null datatype is used to represent an intentional empty value.

For example -

```
let data : null = null;
console.log(data);
```

f. **Undefined:** Undefined datatype is a datatype where a variable is declared but not assigned any value.

For example -

```
let example : undefined
example = undefined
console.log(example);
```

2. Non - Primitive Datatype - Non - primitive datatype in typescript is used to store multiple values and they are mutable in nature.

➢ Object - An object is a real time entity which stores values in the form of key and pair.

```
//? Type Annotation of an Object -

//!  first  approach  of  assigning  a  value  to  an
object. -

let user : {
    username : string,
    userage : number,
    userIsActive : boolean
} = {
```

```
    username : "Ankita",
    userage : 35,
    userIsActive : false
}
console.log(user);

//!  Second  approach  of  assigning  a  value  to  an
object. -

let user : {
    username : string,
    userage : number,
    userIsActive : boolean
}
user = {
    username : "Lalitha",
    userage : 30,
    userIsActive : true
}
console.log(user);

//? Accessing and updating a value in an object -

let user : {
    username : string,
    userage : number,
    userIsActive : boolean
```

```
}
user = {
    username : "Lalitha",
    userage : 30,
    userIsActive : true
}
user.userIsActive = false
console.log(user);

//?  Type  Annotation  of  an  object  with  optional
parameter / optional property-

//? optional parameter - An optional parameter is a
concept  in  typescript  which  is  used  to  make  a
particular property or a parameter as an optional
that means not mandatory. It is denoted with a ?.

let user : {
    username : string,
    userage : number,
    userIsActive ?: boolean
}
user = {
    username : "Lalitha",
    userage : 30,
}
console.log(user);
```

```
//? nested object - A nested object is defined as an object inside another object.

let user : {
    username : string,
    userage : number,
    userIspresent : boolean,
    userAddress : {
        city : string
    }
}
user = {
    username : "Ankita",
    userage : 30,
    userIspresent : true,
    userAddress : {
        city : "Mumbai"
    }
}
console.log(user.userAddress.city);
```

Array - An array is a non primitive datatype which is used to store a collection datatype and it is stored in an [].

//! Type Annotation to an array -

```
let arr : string[] = ["abc","cde","efg0"]
console.log(arr);
```

//! Type annotation to an array where accessing and modification of an element in an array is been done -

```
let arr : string[] = ["abc","cde","efg0"]
console.log(arr[1]);
arr[1] = "XYZ"
console.log(arr);
```

//! Type Annotation to Nested Array -

```
let groups : string[][] = [
    ["ankita","rutuja"],["Rashmi","Sandhaya"]
]
console.log(groups);
```

//! Type Annotation of an array with an object -

```
let student:{
    std_name:string,
    std_rollno:number,
    std_address?:string
}[]
student = [
    {
        std_name:"Shreyas",
        std_rollno:1001,
        std_address:"Bengaluru"
    },
```

```
    {
        std_name:"Sandhya",
        std_rollno:1002,
        std_address:"chennai"
    },
    {
        std_name:"Shweta",
        std_rollno:1003,
    }
]
console.log(student);
```

//! Basic methods of an Array -

```
//! push() - A push() in an array is used to add an
element to the end of the array.

let num:number[] = [10,20,30,40]
console.log(num);
num.push(100)
console.log(num);

//! pop() - A pop() is a method in an array which
is used to remove an element from the last of an
array.
let num : number[] = [100,200,300,400]
console.log(num);
num.pop()
console.log(num);
```

```
//! unshift() - An unshift() in an array is used to
add an element to the start of an array.

let color : string[] =
["red","green","yellow","black","pink"]
console.log(color);
color.unshift("white")
console.log(color);

//! shift() - A shift() in an array is used to
remove the first element from an array.

let color : string[] =
["red","white","yellow","black"]
console.log(color);
color.shift()
console.log(color);
```

//! Advance methods of an Array -

```
//! map() - A map() in an array is an advanced
method in an array which is used to iterate each
and every element of an array and it provides us
with a new array.

// [10,20,30]  X 2 ==> [20,40,60]
```

```
//? syntax -  variable_declaration_type arr_name :
datatype[] = old_array_name.map(callback_function)

let num : number[] = [10,20,30]
let new_num : number[] = num.map((value)=>value*2)
console.log(new_num);

//! filter() - A filter() is an advanced method in
an array which is used to iterate each and every
element in an array based on some filter condition.

let num : number[] = [10,20,30,40]
console.log(num);
let new_num : number[] =
num.filter((value)=>value>10)
console.log(new_num);

//! reduce() - A reduce() is an advanced method of
an array which is used to combine each and every
element of an array into  a single value.

//? reduce - variable_declaration_aray new_array :
datatype[] =
old_array.reduce((accumulator,current_value)=>{//ta
sk to be perform},initial_value)

let num:number[] = [100,200,300]
```

```
console.log(num);
let new_arr:number=
num.reduce((accumulator,current_value)=>{
    return  accumulator + current_value
},0)
console.log(new_arr);
```

Accumelator	current_value	Initial Value
0	100	100
100	200	300
300	300	600

```
//?  function   -  A  function  is  a  non  primitive
datatype which is a reusable block of code which is
used for performing a task.
//!  function  function_name(){//task  to  perform}
function_name()

function demo(a : string, b:number){
    console.log(a+b);
    console.log("this is a typescript session")
}
demo("false",10)
```

```
//? tuple - Tuple is a special datatype in typescript which is used to store a collection of elements.Here the sequence of the elements matter the most.
//! syntax - declaraction_type_of_variable variable : [datatype] = [values]
let data : [string,number,boolean,string] = ["Typescript",7,true,"playwright"]
console.log(data);

//? tuple accessing and updating -
let data : [string,number,boolean,string] = ["Typescript",7,true,"playwright"]
data[3] = 150 //! --> error
console.log(data);

//? tuple with an optional parameter -
let data : [string?, string?, boolean?,number?] = ["a","b",,50]
console.log(data);

//? tuple with a functional parameter -
function demo(a:[number,number,boolean]){
    return [a[0]*10,a[1]*5]    //! --> using array
    return {                   //! --> using object
        p : a[0]*10,
        q : a[1]*5
```

```
    }
}
console.log(demo([10,10,false]));

//? tuple with an array -
let data_arra : [number,number[]][] = [
[10,[1,2,3,4,5,6]],[10,[1,2]],[20,[1,2]],[10,[1,2]]
]
console.log(data_arra);

//? 4. create a function that returns a tuple
(status, order_id)
function order(order_detail:[string,number]){
      return `The order details are like : order
status - ${order_detail[0]} and order id -
${order_detail[1]}.`
}
console.log(order(["successful",1504]));
```

3. Special Datatype - A special datatype is a datatype that doesn't represent normal values like number or string, but are used for type control safety or special behavior.

There are four types in special datatypes : -

1. **Any -** It can store any type of value. It is used for no typing checking, not safe and not used for real projects.

```
let example : any

example = "abc"      //string datatype value
console.log(example);

example = 10         //number datatype value
-- reinitialization

console.log(example);
```

2. **Unknown -** Unknown is a special datatype which is the safer version of any.

```
let response : unknown
response = "TYPESCRIPT"  // storing string
value

response = 50 //storing a number datatype
value -- reinitialization

if(typeof response === "string"){
    console.log("condition is satisfied");
}
else{
    console.log("condition is not
satisfied");
}
```

3. **Void -** Void is a special datatype which is used for the function that does not return anything.

```
//example 1 -
function add(a:number,b:number){
    return a+b
}
console.log(add(10,20));

//example 2 -
function greet(name:string):void{
    console.log(name);
}
greet("Ankita")

//? void datatype is used for the return keyword.
```

4. **Never -** Never is a special datatype which is used when a function never returns anything, function throws an error, infinite loop.

```
//? example 1 -
function error_name():never{
    throw new Error("There is an Error")
}
```

```
error_name()

//? example 2 -
function loop_name():never{
    while(true){
        console.log("this is a loop ...");
    }
}
loop_name()
```

Union - A union type allows a variable to hold multiple possible datatypes. To use union type we have to use union operator which denoted by ('|') pipe operator.

```
//! Union type with variable -
let data : string | number | boolean
data = 80
console.log(data);
data = "Ankita"
console.log(data);
data = true
console.log(data);

//! union type for a function parameter -
function user_id(id:string|number){
    console.log("user_id: ",id);
}
```

```
user_id("ABC1001")
user_id(1001)

//! Union type Narrowing - It means that we need to
check the type before using it.

function narrow(data:string | number){
    if(typeof data === "string" ){
//string === string
        console.log("the condition is satisfied");
    }
    else{
        console.log("the condition is not
satisfied.");
    }
}
narrow("Ankita")
```

```
//! union with an array -

let arr :string[] = ["a","b",true]  --> homogenous
datatype array
console.log(arr);

let arr : (string|boolean|number)[]
```

```
arr = [true,"hello",false,true,"welcome","hi",90]
//--> heterogenous datatype array.
console.log(arr);
```

```
//! union with an object -
let student:{
    std_id:string|number,
    std_name:string,
    std_phone_no:number,
    std_isPresent : boolean
}
student = {
    std_id:101,
    std_name:"Ankita",
    std_phone_no:1234567890,
    std_isPresent:true
}
console.log(student);
```

Type Alias - It is an approach for defining custom names for the existing types. It improves readability and maintainability.

- ❖ Type alias allow you to give a name to a type so that you can use it easily again and again.
- ❖ You can use type alias for primitive, objects, union and function.
- ❖ It is created using the type keyword.

Syntax -

type alias_name = type_defination;

type → type keyword

alias_name → custom name assigned to type.

type_defination → type structure

When to use Type Alias -

1. When defining union types.
2. When defining primitive alias
3. When creating function types.
4. It is used for clear understanding, easier to understand and is reusable.

```
//! type alias for a variable -
type data = number   //  --> type alias is done
let person1 : data = 50
console.log(person1);
let person2 : data  = 40
console.log(person2);

//! type alias for an object -
type obj = {
    product_id : number,
    product_name : string,
    product_catogery : string
}
```

```
let product1 : obj = {
    product_id:101,
    product_name:"xyz",
    product_catogery:"Electronics"
}
console.log(product1);

let product2 : obj = {
    product_id:102,
    product_name:"abc",
    product_catogery: "Cosmetic"
}
console.log(product2);

//! type alias with a function -
type demo = (a:number,b:number)=>number
let add : demo = (x,y) => x+y
console.log(add(10,20));       // output - 30
let sub : demo = (p,q) => p-q
console.log(sub(50,40));      // output - 10
let mul : demo = (c,d) => c*d
console.log(mul(50,10));      // output - 500
let div : demo = (g,h) => g/h
console.log(div(60,3));
```

```
//! type alias with an array -
type value = (string|number|boolean)[]
let arr1 : value = ["ABC","EFG","IJK"]// collection
of string datatype values.
console.log(arr1);
let arr2 : value = [10,20,30,40] // collection of
number datatype values.
console.log(arr2);
let arr3 : value =
[true,false,true,true]//collection of boolean
datatype values.
console.log(arr3);
let arr4 : value = [10,50,"abc",true] // collection
of multiple datatypes.
console.log(arr4);
```

Intersection - Intersection allows you to combine multiple types into one. It is written using the ampersand (&) symbol.

An object or a variable of an intersection type must satisfy all the requirements of each type it combines.

```
type student = {
    std_id:number,
    std_name:string,
    std_isPresent:boolean
}                           // --> type alias 1
```

```
type teacher = {
    tech_id:number,
    tech_name:string,
    subject:string
}                              // --> type alias 2

type school = student & teacher  // intersection of
type alias 1 and 2

let school1 :school = {
    std_id:20,
    std_name : "ABC",
    std_isPresent:true,
    tech_id:101,
    tech_name:"XYZ",
    subject : "typescript"
}
console.log(school1);
```

// **Multiple Intersection** - In Multiple intersections , we can combine more than two type alias.

```
type person = {
    username : string
}                            // --> type alias 1

type emp = {
```

```
    emp_id : number
}                           // --> type alias 2

type address = {
    city_name : string
}                           // --> type alias 3

type staff = person & emp & address
// --> multiple intersection

let user1 : staff = {
    username : "ABC",
    emp_id : 101,
    city_name : "Mumbai"
}
console.log(user1);

let user2 : staff = {
    username : "XYZ",
    emp_id : 502,
    city_name : "Banglore"
}
console.log(user2);
```

// Intersection with function parameter - In this basically we are accessing all the properties from multiple type alias into a function parameter.

```
type user = {
    username : string
}                              // --> type alias 1

type phone = {
    phone_no : number
 }                             // --> type alias 2

type demo = user & phone     // --> intersection of
type alias 1  & 2

function detail(userdetails : demo ){
    console.log(userdetails.username ,
userdetails.phone_no);
}
detail({
    username : "Ankita",
    phone_no : 1234567890
})

//! intersection with a method -
type log_in = {
    login():void
```

```
}                                   // type alias 1
type log_out = {
    logout():void
}                                 // type alias 2

type useraction = log_in & log_out  // intersection
of two type alias.

let user : useraction = {
    login() {
        console.log("login is Successful");
    },
    logout() {
        console.log("logout is Successful");
    },
}
user.login();
user.logout();

//! Intersection with optional parameter -
type person  = {
    name : string
}                           // --> type alias one
type employee = {
    emp_id : number
}                          //  --> type alias two
```

```
type staff = person & employee  // intersection of
person and employee
let obj : staff = {
    name : "Ankita",
    emp_id : 101
}
console.log(obj);

//! Intersection with union -
type one = {
    name : string
}                    //--> the first type alias
type two = {
    age : number,
    id : number | string
}                   // --> the second type alias
type result = one & two    // --> intersection of
one and two
let obj : result = {
    name : "abc",
    age: 50,
    id : 1001
}
console.log(obj);
```

Type Assertion - Type Assertion is a mechanism that tells the compiler to treat a value as a specific type. It is similar to type casting in other languages but it has no effect at runtime, it is purely a compile time construct used to satisfy the type checker.

```
let num : unknown = 60
console.log(typeof num);

let new_num = num as number
console.log(new_num);
```

Literal Type in TypeScript - A literal type means a variable can hold only one specific value, not just a general datatype like string or number.

```
//!  Number literal -
let num : 40
num = 40
console.log(num);

//! string literal -
let age : "10"
age = "10"
console.log(age);
```

```
//! boolean literal -
let ispresent : true
ispresent = true
console.log(ispresent);

//! literal datatype using a function  -
function theme(settheme : "dark"){
    console.log(settheme);
}
theme("dark")

//! literal datatype using an object -
let user : {
    name:string,
    age:50
}
user = {
    name : "ABC",
    age : 50
}
console.log(user);

//! literal datatype with an union -
let color : "white" | "black" | "red"
//-->annotating literal datatype
color = "black"
```

```
color = "white"
color = "red"
color = "green" //it will throw us an error
console.log(color);
```

Interface - In typescript, interface is used to define the shape or structure of an object. It will tell the compiler what properties and types an object should have.

```
// Syntax -
// interface object_name {
// properties : datatype
// }
```

```
//! interface with an object -
interface obj{
    obj_name : string,
    obj_isActive : boolean,
    obj_age: number
}
let user1 : obj = {
    obj_name:"ABC",
    obj_isActive:true,
    obj_age:60
}
console.log(user1);
let user2 : obj = {
```

```
    obj_name : "XYZ",
    obj_age:60,
    obj_isActive : false
}
console.log(user2);
```

```
//! interface with optional parameter -
interface person {
    person_age : number,
    person_name : string,
    person_address ?:string
}
let user1 : person = {
    person_name : "PQR",
    person_age : 20,
    person_address : "mumbai"
}
console.log(user1);
let user2 : person = {
    person_name : "ABC",
    person_age : 50
}
console.log(user2);
```

```
//! interface with function -
interface data {
```

```
    (a:number,b:number):number
}
let sum : data = (x,y) => x+y          // used arrow
function
console.log(sum(20,30));
let sub : data  = (p,q) => p-q
console.log(sub(50,30));
```

```
//! interface with method -
interface greet{
    name : string,
    greet_msg(): void
}
let message : greet = {
    name : "Ankita",
    greet_msg() {
        console.log(`my name is ${this.name}`);
    },
}
console.log(message .greet_msg());
//! interface with readonly property / keyword -
interface data {
    readonly user_id : number,
    user_name:string
}
let user : data = {
    user_id : 101,
```

```
    user_name : "ABC"
}
console.log(user.user_id  =  1001);      //?  updating
and  modification  is  not  allowed  to  a  readonly
property
//! merging two  interface into one object -
interface obj1 {
    p1:string
}                  //? interface 1

interface obj2{
    p2:number
}                //? interface 2

let obj3 : obj1 & obj2 = {     //? merging interface
1 & interface 2
    p1:"orange",
    p2 : 500
}
console.log(obj3);
```

Generics - Generics allow us to write reusable and flexible code that works with different datatypes while keeping type safety. Instead of using any , we use type parameters.

```
// ! Generics with function -
function example<T>(value:T){
```

```
    return value
}
console.log(example<string>("ABC"));
console.log(example<number>(60));
```

```
//! with multiple generics -
function props <K,V>(name:K,age:V){
    return {
        name,
        age
    }
}
console.log(props<string,number>("Ankita",30));
console.log(props<string,string>("ABC","30"));
```

```
//! generic with an array -
function arr<T>(value:T[]){
    console.log(value[0]);
}
arr<number>([10,20,30,40])
```

```
//! generic with interface -
interface obj<T>{
    value : T
}
```

```
let obj1 :obj<number> = {
    value : 100
}
console.log(obj1);
let obj2 : obj<string> = {
    value : "ABC"
}
console.log(obj2);
let obj3 : obj<boolean> = {
    value : true
}
console.log(obj3);
```

OOPs - OOPs stands for object oriented programming. It is a way of writing the code using class and object. It helps to organize the code and make it reusable and easier to manage.

Inheritance - Inheritance in TypeScript means one class can use the properties and methods of another class.

```
class Animal{                  // --> parent class
    sound(){            // --> method for parent class
        console.log(" It is an Animal Sound");
    }
}
```

```
class cat extends Animal{              // --> child
class inheriting method from parent class
    cat_sound(){     // --> method for child class
        console.log("cat sound like meow");
    }
}
let obj = new cat()
obj.cat_sound()
obj.sound()
```

Method Overriding - It means a child class provides its own implementation of a parent class method.

```
class Animal {        // --> parent class
    sound(){         // --> method for parent class
        console.log("It is an Animal sound");
    }
}
class cat extends Animal{                  // --> child
class inheriting method from parent class
     sound(){                              // --> child
class method overriding parent class method
        console.log("cat sound is like a meow.");
    }
}
let obj = new cat()
obj.sound()
```

Method overloading - In Method Overloading there is the same name for the method but different parameters to it.

```
class calculator{
    add(a:number,b:number):number;
// --> first condition where two parameter
    add(a:number,b:number,c:number):number
// --> second condition where there are three
parameter
    add(a:number,b:number,c?:number) : number {
        if(c!==undefined) {
            return a+b+c
        }
        else{
            return a+b
        }
    }
}

let obj = new calculator()
console.log(obj.add(10,20));
// --> else condition should be executed
console.log(obj.add(10,20,30));
// --> if condition should satisfy
console.log(obj.add(10,20,30,40));
// --> getting an error
console.log(obj.add(10));
```

```
// --> getting an error
```

Access Modifier -

An access modifier controls who can access properties and methods in a class. There are three types of access modifier -

1. **Public -**

   A Public Access Modifier can be accessible anywhere inside a class as well as outside the class.

```
// class student{
//     public name:string
//     constructor(username:string){
//         this.name = username
//     }
//                                 display(){
//--> inside the class accessibility.
//                     console.log("This  is  a
Username:",this.name);
//     }
// }

// class person extends student{
//     greet(){
//         console.log("hi");
//     }
// }
```

```
// let res = new student("Ankita")
// let res2 = new person("ANkita")
// console.log(res2.name);
// res.display()
//  console.log(res.name);                    //-->
outside the class accessibility.
```

2. **Private -**

   A Private Access Modifier is accessible only inside the class and not outside the class.

```
// class student{
//     private name:string
//     constructor(username:string){
//         this.name = username
//     }
//     private display(){
//                 console.log("This  is  my  name:
",this.name);        //--> accessibility inside
the class
//     }
// }

// class person extends student{
//     display(){
//         console.log(this.name);
```

```
//     }
// }
// let res4 = new person("Ankita")
// res4.display()
// let res3 = new student("Ankita")
// res3.display()
// console.log(res3.name);
```

3. **Protected -**

   A Protected Access Modifier is accessible inside the class, it can be accessible inside the child class but cannot be accessed outside the class.

```
// class person{
//     protected salary:number
//     constructor(sal:number){
//         this.salary = sal
//     }

// }

// class employee extends person{
//     salary_update(){
//         console.log("salary:",this.salary);
//     }
// }
```

```
// let res5 = new employee(500000)
// res5.salary_update()
// res5.salary
```

Encapsulation -

An Encapsulation in OOPS is used for hiding internal data and controlling the access using the methods.

```
// class Bank{
//     private Amount : number = 0
//     deposite(amt:number){
//         this.Amount += amt;
//     }
//     Balance(){
//         return this.Amount;
//     }
// }

// let res6 = new Bank();
// res6.deposite(1000)
// console.log(res6.Balance());
```